CUADERNO DE PRÁCTICAS // DESARROLLO WEB

D3 · UD2
SPRING BOOT

D3-UD2

Kerin Aguilera

JAVA 21 · SPRING BOOT · OCTUBRE 2026

PRIMER FORMULARIO

creamos el formulario, el primero formulario no tiene hidden, porque es el primero.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>FORMULARIO UNO</title>
</head>
<body>
    <form method="post" action="/procesar" th:action="@{/procesar}" th:object="${tusDatos
        <label>Nombre:</label>
        <input type="text" th:field="*{nombre}" name="nombre">

        <button type="submit" name="contador" value="1" >Enviar</button>
    </form>
</body>
</html>
```

Luego al dar el boton enviar, debos de tener lo siguiente:

Utilizamos en java para visualizar en consola

```
        System.out.println(datos);
        System.out.println(siguiente);
```

```
TusDatos(nombre=Ruvik, Aficciones=[], lugar=null, datosFinales=[])
1
```

✦ SEGUNDO FORMULARIO

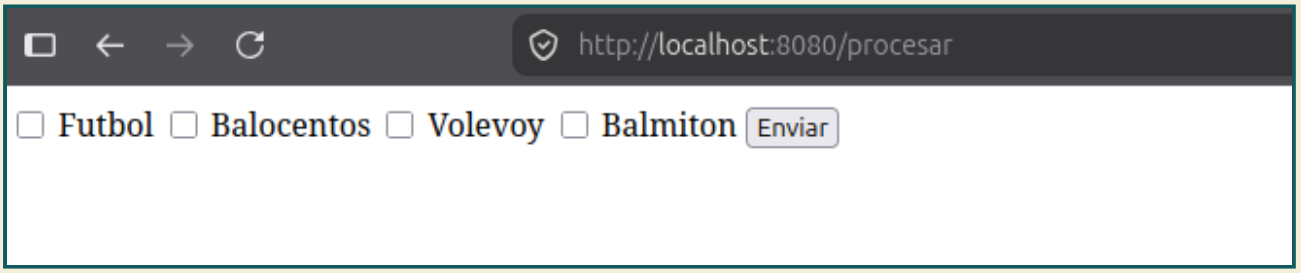
aqui encontramos el primero campo **hidden** donde por medio de un input volvemos a introducir los datos recibidos de la anterior peteción

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>FORMULARIO DOS</title>
</head>
<body>
<form method="post" th:action="@{/procesar}" th:object="${datos}" action="/procesar">
    <input type="checkbox" th:field="*{Aficciones}" name="futbol" value="futbol"> Futbol
    <input type="checkbox" th:field="*{Aficciones}" name="balocentos" value="balocentos">
    <input type="checkbox" th:field="*{Aficciones}" name="volevoy" value="volevoy"> Vole
    <input type="checkbox" th:field="*{Aficciones}" name="balmiton" value="balmiton"> Ba
    <button type="submit" name="contador" value="1" >Enviar</button>


    <input type="hidden" th:field="*{nombre}" />

</form>
</body>
</html>
```

El formulario se ve



y en esta segundo operación nuestro break de mostrar datos debe enseñar lo siguiente:

```
TusDatos(nombre=Ruvik, Aficciones=[], lugar=null, datosFinales=[])
1
TusDatos(nombre=Ruvik, Aficciones=[balocentos, balmiton], lugar=null, datosFinales=[])
2
```

como vemos los datos van recdordandose segun va la Marcha

TERCER FORMULARIO

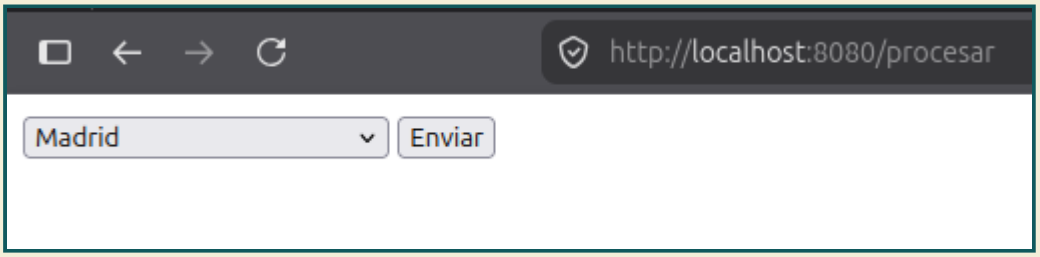
Ahora aqui nos llevar dos campos hidden, donde volvemos arrastrar los datos anteriores para guardarlos con los nuevos:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
<form method="post" th:action="@{/procesar}" th:object="${datos}" action="/procesar">
    <select name="lugar" th:field="*{lugar}">
        <option value="" disabled selected >-- Selecciona una opción --</option>
        <option value="Madrid">Madrid</option>
        <option value="Sevilla">Sevilla</option>
        <option value="Malaga" >Malaga</option>
    </select>

    <button type="submit" name="contador" value="1" >Enviar</button>

    <input type="hidden" th:field="*{nombre}" />
    <input type="hidden" th:field="*{Aficciones}"/>
</form>
</body>
</html>
```

se veria de la siguiente forma el formulario:



y Al dar enviar deberiamos tener todos los datos guardados:

```
TusDatos(nombre=Ruvik, Aficciones=[], lugar=null, datosFinales=[])
1
TusDatos(nombre=Ruvik, Aficciones=[balocentos, balmiton], lugar=null, datosFinales=[])
2
TusDatos(nombre=Ruvik, Aficciones=[balocentos, balmiton], lugar=Sevilla, datosFinales=[])
3
```

tenemos todo ahora solo queda visualizar el resultado

RESULTADO FORMULARIO

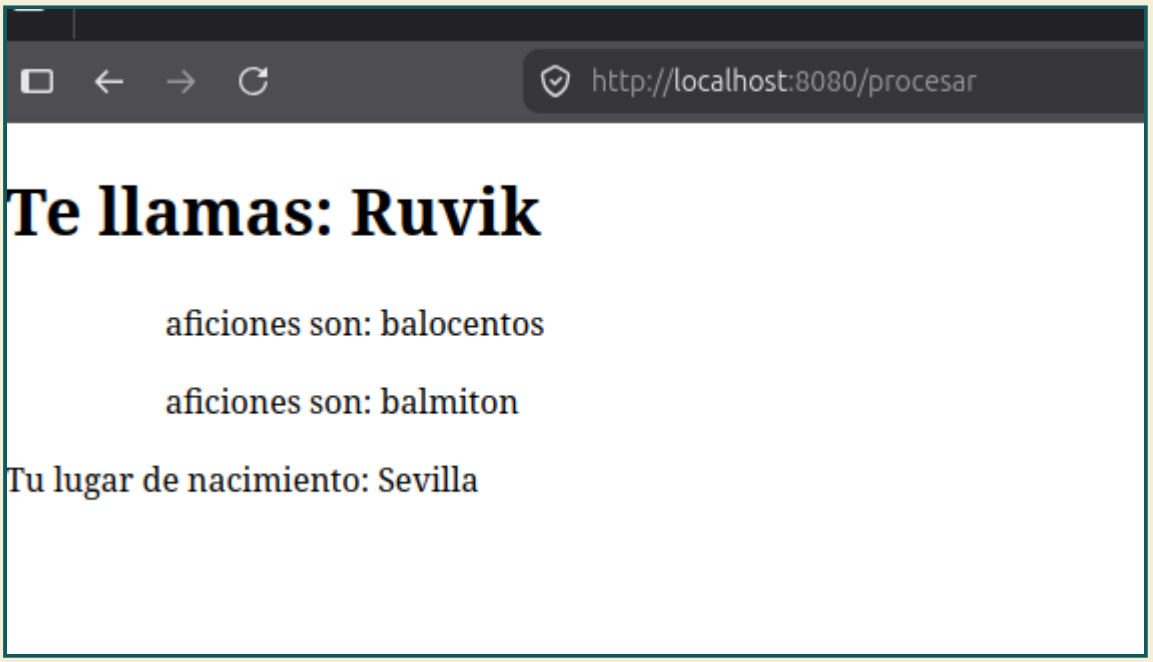
aqui es donde todo lo que acarreamos se tiene que visualizar, utilizamos las antaciones `th:text` y `th:each` parar hacerlo

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>MOSTRAR DATOS</title>
</head>
<body>
    <h1 th:text="| Te llamas: ${datos.nombre} |"></h1>

    <ol th:each="aficcion: ${datos.Aficciones}">
        <ul  th:text="|aficiones son: ${aficcion}|" ></ul>
    </ol>

<p th:text="| Tu lugar de nacimiento: ${datos.lugar} |"></p>
</body>
</html>
```

gracias a esto podemos visualizar los datos de la siguiente forma:



LOGICA

de tras de todos esto, hay un @PostMapping que va eligiendo que formulario se muestra y cual no,

yo lo realizada esperando un datos en `@RequestParam` que le llamae contador y es el el `value=""` del boton de enviar, este decide, cual entra y cual no.

La cuestión importante que estoy usando un contador global, clarmaente podría hacerlo por campos hidden, podría intentar y corregir este fallo que tengo una solución rapida para hacer todo rapido.

la logica es la siguiente:

```java
package ruvik.d3ud2.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import ruvik.d3ud2.TusDatos;

@Controller
public class ControladorFormulariosEncadenado {
    static int siguiente = 0;


    @GetMapping("/formulario1")
    public String formulario1(@ModelAttribute TusDatos tusDatos, Model model) {
        model.addAttribute("tusDatos", tusDatos);
        return "Formulario1";
    }


    @GetMapping("/formulario2")
    public String formulario2(@ModelAttribute TusDatos tusDatos, Model model) {
        model.addAttribute("tusDatos", tusDatos);
        return "Formulario2";
    }

    @PostMapping("/procesar")
    public String procesarDatos(@ModelAttribute TusDatos datos,
                                @RequestParam("contador") String contador,
                                Model modelo) {
        siguiente += Integer.parseInt(contador);
        System.out.println(datos);
        System.out.println(siguiente);

        if (siguiente == 1) {
            modelo.addAttribute("datos", datos);
            return "/Formulario2";
        }

        if (siguiente == 2) {
            modelo.addAttribute("datos", datos);
            return "/Formulario3";
        }


        modelo.addAttribute("datos", datos);
        return "/Resultado";
    }
}
```

yo utilice Modelatribute asi que si tengo un pojo donde lo guardo e introduzco los datos:

```
package ruvik.d3ud2;

import lombok.Data;

import java.util.ArrayList;
@Data
public class TusDatos {
    private String nombre;
    private ArrayList<String> Aficciones = new ArrayList<>();
    private String lugar;
    private ArrayList<String> datosFinales = new ArrayList<>();


    public TusDatos() {
    }
}
```

es pequeño por utilizo el paquete Loombok para evitar escribir geters y setters